

SKILL WEEK-05

1. To Change Directories, Validate Paths, Update Working Directory, Handle Errors, Maintain Previous Directory, Test Navigation

CODE:

```
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <limits.h>

int main()
{
    char path[PATH_MAX];
    char prev[PATH_MAX];

    getcwd(prev, sizeof(prev));

    printf("Current Directory: %s\n", prev);

    printf("Enter directory path: ");
    scanf("%s", path);

    if(chdir(path) == 0)
    {
        char current[PATH_MAX];
        getcwd(current, sizeof(current));

        printf("Directory changed successfully.\n");
        printf("New Directory: %s\n", current);
    }
    else
    {
        perror("Error");
    }

    return 0;
}
```

RESULT:

```
root@SRIVALLI:~# nano week5_cd.c
root@SRIVALLI:~# gcc week5_cd.c -o week5_cd
root@SRIVALLI:~# ./week5_cd
Current Directory: /root
Enter directory path: Documents
Error: No such file or directory
root@SRIVALLI:~# ./week5_cd
Current Directory: /root
Enter directory path: /home/student/Documents
Error: No such file or directory
root@SRIVALLI:~# ./week5_cd
Current Directory: /root
Enter directory path: abc123
Error: No such file or directory
```

REPORT:

In this experiment, a program is developed to change directories in Linux using the `chdir()` system call. The program accepts a directory path from the user, validates whether the path exists, and updates the current working directory. If the specified

path is invalid, an error message is displayed. The current and updated working directories are shown using `getcwd()`. The program also demonstrates error handling and directory navigation concepts in Linux. This experiment helps in understanding file system navigation, path validation, and working directory management in an operating system

2. TO Identify Built-ins, Create Dispatch Table, Execute In- Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic.

CODE:

```
GNU nano 7.2 dispatch_shell.c *
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

void cmd_help()
{
    printf("Available commands: help, hello, exit\n");
}

void cmd_hello()
{
    printf("Hello User!\n");
}

int main()
{
    char command[50];

    while(1)
    {
        printf("myshell> ");
        scanf("%s", command);

        if(strcmp(command, "help") == 0)
        {
            cmd_help();
        }
        else if(strcmp(command, "hello") == 0)
        {
            cmd_hello();
        }
        else if(strcmp(command, "exit") == 0)
        {
            printf("Exiting shell...\n");
            break;
        }
        else
    }
```

RESULT:

```
root@SRIVALLI:~# nano dispatch_shell.c
root@SRIVALLI:~# gcc dispatch_shell.c -o dispatch_shell
root@SRIVALLI:~# ./dispatch_shell
myshell> help
Available commands: help, hello, exit
myshell> hello
Hello User!
myshell> abc
Invalid command
myshell> |
```

REPORT:

In this experiment, a simple shell program is developed to identify and execute built-in commands using a dispatch mechanism. The program accepts user commands and checks them against predefined commands such as help, hello, and exit. Valid commands are executed within the same process, while invalid commands are handled by displaying an error message. The shell maintains its state by continuously accepting commands until the user chooses to exit. This experiment helps in understanding command dispatch logic, built-in command execution, state management, and error handling in operating systems.

3. To Retrieve Current Directory, Display Path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behavior

CODE:

```
GNU nano 7.2 pwd_exit.c *
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <limits.h>

int main()
{
    char cwd[PATH_MAX];
    char command[20];

    while(1)
    {
        getcwd(cwd, sizeof(cwd));
        printf("Current Directory: %s\n", cwd);

        printf("Enter command (pwd/exit): ");
        scanf("%s", command);

        if(strcmp(command, "pwd") == 0)
        {
            printf("Path: %s\n", cwd);
        }
        else if(strcmp(command, "exit") == 0)
        {
            printf("Exiting program...\n");
            break;
        }
        else
        {
            printf("Invalid command\n");
        }
    }

    return 0;
}
```

RESULT:

```
root@SRIVALLI:~# nano pwd_exit.c
root@SRIVALLI:~# gcc pwd_exit.c -o pwd_exit
root@SRIVALLI:~# ./pwd_exit
Current Directory: /root
Enter command (pwd/exit): pwd
Path: /root
Current Directory: /root
Enter command (pwd/exit): exit
Exiting program...
root@SRIVALLI:~# |
```

REPORT:

In this experiment, a program is developed to retrieve and display the current working directory using the `getcwd()` function. The program accepts user commands such as `pwd` to display the current directory path and `exit` to terminate the program. It continuously processes user requests, maintains the current state, and handles invalid commands appropriately. Before exiting, the program performs normal cleanup by terminating the execution loop. This experiment helps in understanding directory management, command processing, state maintenance, and program termination in a Linux environment.

4. To Parse Export Syntax, Update Environment Variables, Validate Names, Handle Existing Variables, Export to Child Processes, Test Functionality

CODE:

```

int validName(char *name)
{
    if(!isalpha(name[0]) && name[0] != '_')
        return 0;

    for(int i = 1; name[i]; i++)
    {
        if(!isalnum(name[i]) && name[i] != '_')
            return 0;
    }
    return 1;
}

int main()
{
    char var[50], value[50];

    printf("Enter variable name: ");
    scanf("%s", var);

    if(!validName(var))
    {
        printf("Invalid variable name!\n");
        return 1;
    }

    printf("Enter value: ");
    scanf("%s", value);

    setenv(var, value, 1);

    printf("Environment Variable Set Successfully\n");
    printf("%s=%s\n", var, getenv(var));

    return 0;
}

```

RESULT:

```

root@SRIVALLI:~# nano export_env.c
root@SRIVALLI:~# gcc export_env.c -o export_env
root@SRIVALLI:~# ./export_env
Enter variable name: SRIVALLI
Enter value: 200
Environment Variable Set Successfully
SRIVALLI=200
root@SRIVALLI:~# |

```

REPORT:

In this experiment, a program is developed to parse and implement the export command functionality in Linux. The program accepts an environment variable name and value from the user, validates the variable name, and updates the environment using the setenv() function. If the variable already exists, its value is updated. Invalid variable names are rejected with an error message. The environment variable can then be accessed by child processes, demonstrating environment inheritance. This experiment helps in understanding environment variable management, validation, process environments, and export functionality in operating systems.